

IDEAS - Institute of Data Engineering, Analytics and Science Foundation

Summer Internship Program 2026

Project Title: Exploratory Data Analysis with EPC Industry Data

Project Notebook

Created by: Koulika Paul
Designation: Project Linked Associate Research Engineer in Statistics

Question 1: Load Dataset and Basic Exploration (1 Mark)

Import `pandas` as `pd` and `numpy` as `np`. Download the dataset `EPC_Industry_Analytics_Demo.xlsx` from <https://drive.google.com/drive/folders/1gieHICVDBbUKMZiSF4YRQMAJic-w50JM?usp=sharing>. Load the dataset `EPC_Industry_Analytics_Demo.xlsx` into a DataFrame named `df`.

Print the shape of the dataset, its data types, and the count of missing values for each column.

Expected Output: The shape tuple, the data types of columns, and the sum of missing values per column.

```
import pandas as pd
import numpy as np

df = pd.read_excel("EPC_Industry_Analytics_Demo.xlsx")

print("Shape:")
print(df.shape)

print("\nData Types:")
print(df.dtypes)

print("\nMissing Values:")
print(df.isnull().sum())
```

```
Shape:
(1200, 14)

Data Types:
Date                datetime64[ns]
Project_ID          object
Region              object
Project_Type         object
Contractor_Tier      object
Planned_Budget_K     float64
Actual_Cost_K        float64
Budget_Variance_Pct  float64
Sensor_Vibration_mm_s float64
Sensor_Temp_C        float64
Efficiency_Score     float64
Material_Price_Index float64
Unnamed: 12          float64
Unnamed: 13          object
dtype: object

Missing Values:
Date                0
Project_ID          0
Region              0
Project_Type        0
Contractor_Tier     60
Planned_Budget_K    0
Actual_Cost_K       0
Budget_Variance_Pct 0
Sensor_Vibration_mm_s 0
Sensor_Temp_C       0
Efficiency_Score    0
Material_Price_Index 0
Unnamed: 12        1200
Unnamed: 13        1199
dtype: int64
```

Question 2: Duplicate Record Analysis (1 Mark)

Find the duplicate rows in the DataFrame `df` and print the total number of duplicate rows.

Expected Output: A print statement showing the count of duplicate rows.

```
duplicate_count = df.duplicated().sum()

print("Total duplicate rows:", duplicate_count)

Total duplicate rows: 0
```

Question 3: Missing Value Treatment (2 Marks)

First, replace any occurrences of blank spaces `' '`, empty strings `''`, `'None'`, and `'-'` with `np.nan` across the entire DataFrame. Next, fill the missing values in all **numeric columns** with the median of their respective columns. Finally, print the first 5 rows of the DataFrame to verify the changes.

Expected Output: A table showing the first 5 rows of the modified DataFrame.

```
# Replace blank values with NaN
df.replace([' ', '', 'None', '-'], np.nan, inplace=True)

# Select numeric columns
numeric_cols = df.select_dtypes(include=np.number).columns

# Fill missing values with median
for col in numeric_cols:
    df[col].fillna(df[col].median(), inplace=True)

# Display first 5 rows
print(df.head())
```

	Date	Project_ID	Region	Project_Type	Contractor_Tier	\
0	2023-01-01	PROJ-1000	North America	Renewable Energy	Tier 3	
1	2023-01-01	PROJ-1001	LATAM	Industrial Plant	Tier 1	
2	2023-01-01	PROJ-1002	APAC	Oil & Gas	Tier 1	
3	2023-01-01	PROJ-1003	North America	Industrial Plant	Tier 2	
4	2023-01-02	PROJ-1004	North America	Renewable Energy	Tier 2	

	Planned_Budget_K	Actual_Cost_K	Budget_Variance_Pct	\
0	2032.718750	2095.845956	3.105555	
1	822.703500	839.771030	2.074566	
2	2343.330027	2501.768017	6.761232	
3	1900.478728	1966.592256	3.478783	
4	3547.042879	3508.019596	-1.100164	

	Sensor_Vibration_mm_s	Sensor_Temp_C	Efficiency_Score	\
0	4.775256	63.900729	64.144031	
1	1.779238	46.502858	83.031245	
2	2.303336	50.985558	81.513558	
3	3.341599	59.448200	72.523566	
4	3.501433	59.905765	70.703579	

	Material_Price_Index	Unnamed: 12	Unnamed: 13
0	100.000000	NaN	NaN
1	100.058382	NaN	NaN
2	100.116760	NaN	NaN
3	100.175133	NaN	NaN
4	100.233497	NaN	NaN

```
/tmp/ipykernel_1960/504516986.py:9: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are sett

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df

df[col].fillna(df[col].median(), inplace=True)
```

Question 4: Statistical Summary (1 Mark)

Generate and display a complete statistical summary of the DataFrame `df`, including all columns (both numeric and categorical).

Hint: Use the `.describe(include='all')` method.

Expected Output: A comprehensive statistical summary table.

Complete statistical summary
df.describe(include='all')

	Date	Project_ID	Region	Project_Type	Contractor_Tier	Planned_Budget_K	Actual_Cost_K	Budget_Variance_Pct	Sensor_Vibration_mm_s
count	1200	1200	1200	1200	1140	1200.000000	1200.000000	1200.000000	1200
unique	NaN	1200	4	4	3	NaN	NaN	NaN	NaN
top	NaN	PROJ-2183	LATAM	Renewable Energy	Tier 2	NaN	NaN	NaN	NaN
freq	NaN	1	317	314	574	NaN	NaN	NaN	NaN
mean	2023-05-30 12:00:00	NaN	NaN	NaN	NaN	2750.430575	2987.503387	8.482366	0.047613
min	2023-01-01 00:00:00	NaN	NaN	NaN	NaN	500.052356	508.320284	-12.012275	-0.006899
25%	2023-03-16 18:00:00	NaN	NaN	NaN	NaN	1650.494262	1801.340050	3.374540	0.019985

Question 5: Correlation Analysis (2 Marks)

Calculate the correlation matrix for all numeric columns in the DataFrame `df` and print the resulting matrix.

Hint: Select numeric columns first using `.select_dtypes(include=np.number)`, then apply `.corr()`.

Expected Output: A correlation matrix showing the linear relationship between numeric features.

Select numeric columns
numeric_df = df.select_dtypes(include=np.number)

Correlation matrix
correlation_matrix = numeric_df.corr()

Display result
print(correlation_matrix)

	Planned_Budget_K	Actual_Cost_K	Budget_Variance_Pct	\
Planned_Budget_K	1.000000	0.987637	0.039850	
Actual_Cost_K	0.987637	1.000000	0.181439	
Budget_Variance_Pct	0.039850	0.181439	1.000000	
Sensor_Vibration_mm_s	0.047613	0.048364	0.016270	
Sensor_Temp_C	-0.006899	-0.010928	-0.006320	
Efficiency_Score	0.019985	0.023992	0.006594	
Material_Price_Index	-0.000661	-0.007159	-0.035049	
Unnamed: 12	NaN	NaN	NaN	
	Sensor_Vibration_mm_s	Sensor_Temp_C	Efficiency_Score	\
Planned_Budget_K	0.047613	-0.006899	0.019985	
Actual_Cost_K	0.048364	-0.010928	0.023992	
Budget_Variance_Pct	0.016270	-0.006320	0.006594	
Sensor_Vibration_mm_s	1.000000	0.707830	-0.697970	
Sensor_Temp_C	0.707830	1.000000	-0.963516	
Efficiency_Score	-0.697970	-0.963516	1.000000	
Material_Price_Index	0.013068	0.021116	-0.012720	
Unnamed: 12	NaN	NaN	NaN	
	Material_Price_Index	Unnamed: 12		
Planned_Budget_K	-0.000661	NaN		
Actual_Cost_K	-0.007159	NaN		
Budget_Variance_Pct	-0.035049	NaN		
Sensor_Vibration_mm_s	0.013068	NaN		
Sensor_Temp_C	0.021116	NaN		
Efficiency_Score	-0.012720	NaN		
Material_Price_Index	1.000000	NaN		
Unnamed: 12	NaN	NaN		

Question 6: Outlier Detection using IQR (3 Marks)

Write a loop to calculate the Interquartile Range (IQR) for each numeric column in `df`. Define outliers as values below $Q1 - 1.5 * IQR$ or above $Q3 + 1.5 * IQR$. Count the number of outliers for each numeric column and store them in a dictionary named `outlier_summary`. Print this dictionary.

Expected Output: A dictionary showing the column names as keys and their respective outlier counts as values.

```
# Select numeric columns
numeric_cols = df.select_dtypes(include=np.number).columns

# Dictionary to store outlier counts
outlier_summary = {}

# Loop through each numeric column
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]

    outlier_summary[col] = len(outliers)

# Print dictionary
print(outlier_summary)
```

```
{'Planned_Budget_K': 0, 'Actual_Cost_K': 0, 'Budget_Variance_Pct': 3, 'Sensor_Vibration_mm_s': 41, 'Sensor_Temp_C': 32, 'Efficie
```

Question 7: Group-Based Aggregation (2 Marks)

Select the first categorical (object) column in the DataFrame. Group the data by this categorical column and calculate the `mean`, `median`, and `sum` for all numeric columns. Display the first 5 rows of the aggregated results.

Hint: Use `.groupby()` combined with `.agg(['mean', 'median', 'sum'])`.

Expected Output: A table showing the first 5 rows of grouped statistical aggregations.

```
# Find the first categorical (object) column
cat_col = df.select_dtypes(include='object').columns[0]

# Select numeric columns
numeric_cols = df.select_dtypes(include=np.number).columns

# Group and aggregate
grouped_result = df.groupby(cat_col)[numeric_cols].agg(['mean', 'median', 'sum'])

# Display first 5 rows
print(grouped_result.head())
```

	Planned_Budget_K		Actual_Cost_K		
	mean	median	sum	mean	
Project_ID					
PROJ-1000	2032.718750	2032.718750	2032.718750	2095.845956	
PROJ-1001	822.703500	822.703500	822.703500	839.771030	
PROJ-1002	2343.330027	2343.330027	2343.330027	2501.768017	
PROJ-1003	1900.478728	1900.478728	1900.478728	1966.592256	
PROJ-1004	3547.042879	3547.042879	3547.042879	3508.019596	

	Budget_Variance_Pct				
	median	sum	mean	median	sum
Project_ID					
PROJ-1000	2095.845956	2095.845956	3.105555	3.105555	3.105555
PROJ-1001	839.771030	839.771030	2.074566	2.074566	2.074566
PROJ-1002	2501.768017	2501.768017	6.761232	6.761232	6.761232
PROJ-1003	1966.592256	1966.592256	3.478783	3.478783	3.478783
PROJ-1004	3508.019596	3508.019596	-1.100164	-1.100164	-1.100164

	Sensor_Vibration_mm_s	...	Sensor_Temp_C	Efficiency_Score	
	mean	...	sum	mean	
Project_ID					
PROJ-1000	4.775256	...	63.900729	64.144031	
PROJ-1001	1.779238	...	46.502858	83.031245	
PROJ-1002	2.303336	...	50.985558	81.513558	

```

PROJ-1003      3.341599 ...      59.448200      72.523566
PROJ-1004      3.501433 ...      59.905765      70.703579

      Material_Price_Index
      median      sum      mean      median      sum \
Project_ID
PROJ-1000      64.144031      64.144031      100.000000      100.000000      100.000000
PROJ-1001      83.031245      83.031245      100.058382      100.058382      100.058382
PROJ-1002      81.513558      81.513558      100.116760      100.116760      100.116760
PROJ-1003      72.523566      72.523566      100.175133      100.175133      100.175133
PROJ-1004      70.703579      70.703579      100.233497      100.233497      100.233497

      Unnamed: 12
      mean      median      sum
Project_ID
PROJ-1000      NaN      NaN      0.0
PROJ-1001      NaN      NaN      0.0
PROJ-1002      NaN      NaN      0.0
PROJ-1003      NaN      NaN      0.0
PROJ-1004      NaN      NaN      0.0

[5 rows x 24 columns]

```

Question 8: Feature Engineering (2 Marks)

Identify the first two numeric columns in the DataFrame. Create a new feature named `New_Feature` by dividing the first numeric column by the (second numeric column + 1). Display the first 5 rows of the DataFrame showing this new feature.

Expected Output: The first 5 rows of the DataFrame containing the newly engineered `New_Feature` column.

```

# Find the first two numeric columns
numeric_cols = df.select_dtypes(include=np.number).columns

first_col = numeric_cols[0]
second_col = numeric_cols[1]

# Create new feature
df['New_Feature'] = df[first_col] / (df[second_col] + 1)

# Display first 5 rows
print(df.head())

```

```

      Date Project_ID      Region      Project_Type Contractor_Tier \
0 2023-01-01 PROJ-1000 North America Renewable Energy      Tier 3
1 2023-01-01 PROJ-1001 LATAM Industrial Plant      Tier 1
2 2023-01-01 PROJ-1002 APAC Oil & Gas      Tier 1
3 2023-01-01 PROJ-1003 North America Industrial Plant      Tier 2
4 2023-01-02 PROJ-1004 North America Renewable Energy      Tier 2

      Planned_Budget_K      Actual_Cost_K      Budget_Variance_Pct \
0      2032.718750      2095.845956      3.105555
1      822.703500      839.771030      2.074566
2      2343.330027      2501.768017      6.761232
3      1900.478728      1966.592256      3.478783
4      3547.042879      3508.019596      -1.100164

      Sensor_Vibration_mm_s      Sensor_Temp_C      Efficiency_Score \
0      4.775256      63.900729      64.144031
1      1.779238      46.502858      83.031245
2      2.303336      50.985558      81.513558
3      3.341599      59.448200      72.523566
4      3.501433      59.905765      70.703579

      Material_Price_Index      Unnamed: 12      Unnamed: 13      New_Feature
0      100.000000      NaN      NaN      0.969417
1      100.058382      NaN      NaN      0.978511
2      100.116760      NaN      NaN      0.936295
3      100.175133      NaN      NaN      0.965891
4      100.233497      NaN      NaN      1.010836

```

Question 9: Ranking Analysis (2 Marks)

Select the first numeric column in your DataFrame. Create a new column called `Rank` which assigns a rank to each row based on the values in this first numeric column in **descending order** (highest value gets rank 1). Display the first 5 rows of the DataFrame showing only the `Rank` column and the first numeric column.

Hint: Use the `.rank(ascending=False)` method.

Expected Output: The first 5 rows of the `Rank` column alongside the target numeric column.

```
# Select the first numeric column
first_numeric_col = df.select_dtypes(include=np.number).columns[0]

# Create rank column (highest value gets rank 1)
df['Rank'] = df[first_numeric_col].rank(ascending=False)

# Display first 5 rows
print(df[[first_numeric_col, 'Rank']].head())
```

	Planned_Budget_K	Rank
0	2032.718750	791.0
1	822.703500	1104.0
2	2343.330027	702.0
3	1900.478728	823.0
4	3547.042879	390.0

Question 10: Conditional Filtering (1 Mark)

Using the first numeric column again, filter the DataFrame `df` to include only those rows where the value in this column is strictly **greater than its mean**. Print the shape of this new conditionally filtered DataFrame.

Expected Output: A tuple showing the number of rows and columns in the filtered DataFrame.

```
# First numeric column
first_numeric_col = df.select_dtypes(include=np.number).columns[0]

# Mean of that column
mean_value = df[first_numeric_col].mean()

# Filter rows greater than mean
filtered_df = df[df[first_numeric_col] > mean_value]

# Print shape
print(filtered_df.shape)
```

(602, 16)

Question 11: Load MNIST Data (3 Marks)

Load the MNIST dataset using `sklearn.datasets.load_digits`. Separate the dataset into features (`X`) and target labels (`y`). Print the shape of the features and the target arrays.

Expected Output: The shape of `X` and `y`.

```
from sklearn.datasets import load_digits

# Load dataset
digits = load_digits()

# Features and target
X = digits.data
y = digits.target

# Print shapes
print("Shape of X:", X.shape)
print("Shape of y:", y.shape)
```

Shape of X: (1797, 64)
Shape of y: (1797,)

Question 12: K-Means Clustering (5 Marks)

Import `KMeans` from `sklearn.cluster`. Initialize and fit the K-Means clustering model on the MNIST features (`X`). Since we know there are 10 digits (0-9), set the number of clusters to 10. Assign the cluster labels to a variable `kmeans_labels`.

Expected Output: A successfully fitted K-Means model and the cluster labels array.

```
from sklearn.cluster import KMeans

# Create K-Means model with 10 clusters
kmeans = KMeans(n_clusters=10, random_state=42)

# Fit model and get cluster labels
kmeans_labels = kmeans.fit_predict(X)

# Display first 10 labels
print(kmeans_labels[:10])
```

[5 7 7 6 4 9 2 0 8 9]

Question 13: F1 Score Evaluation for Clustering (5 Marks)

Evaluate the clustering performance using the F1 score. Since K-Means assigns arbitrary cluster labels, you will first need to map each cluster label to the most frequent true label in that cluster. After mapping the labels, calculate and print the macro-averaged F1 score using `sklearn.metrics.f1_score`.

Expected Output: The calculated F1 score of the clustering.

```
from sklearn.metrics import f1_score
import numpy as np

# Map each cluster to the most common true label
mapped_labels = np.zeros_like(kmeans_labels)

for cluster in np.unique(kmeans_labels):
    mask = (kmeans_labels == cluster)
    true_labels = y[mask]

    most_common_label = np.bincount(true_labels).argmax()
    mapped_labels[mask] = most_common_label

# Calculate F1 score
f1 = f1_score(y, mapped_labels, average='macro')

print("F1 Score:", f1)
```

F1 Score: 0.8627854786352394